Visitor

Visitor · Pretty-print / evaluate expression AST · Build with Waqar

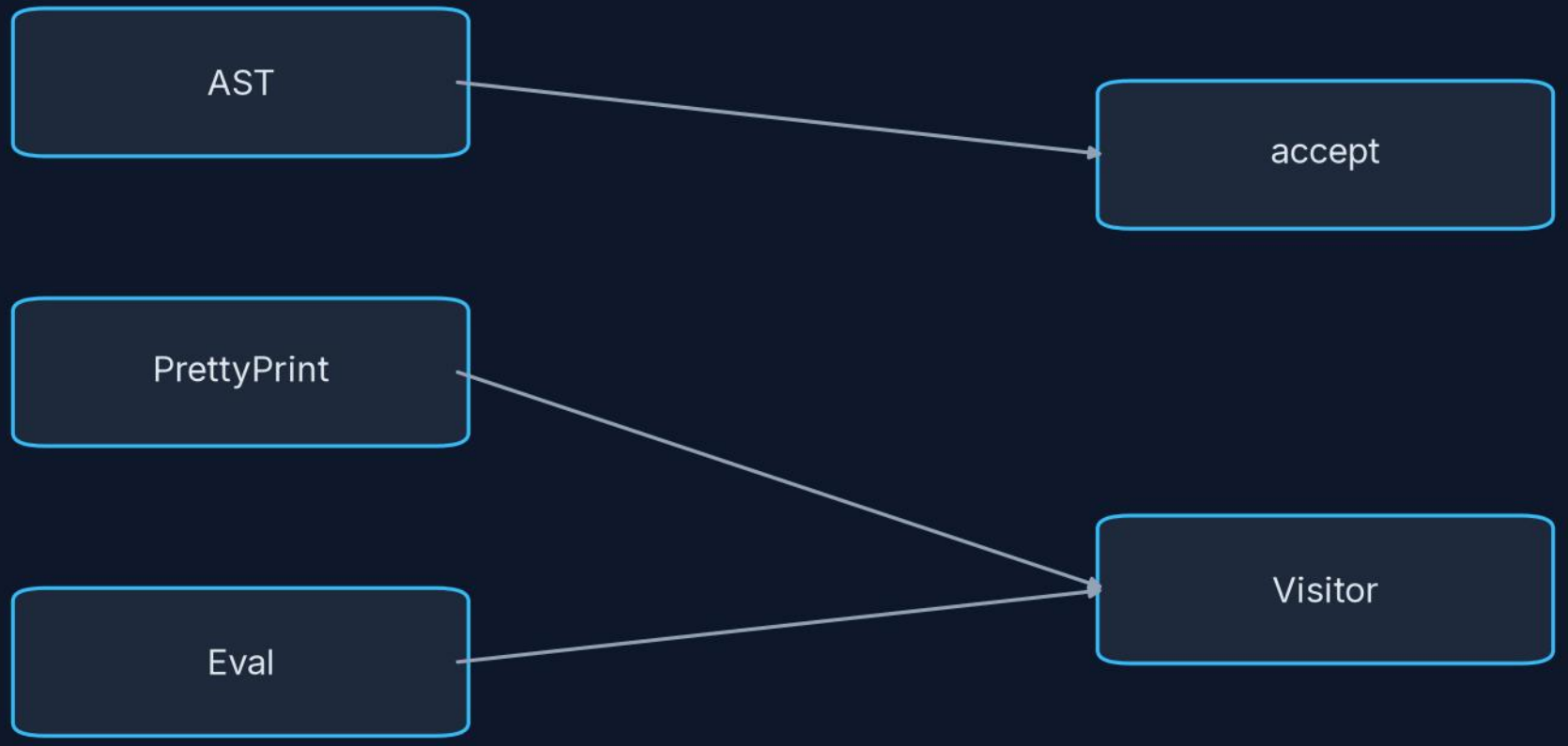
Today's Hook

- Problem: Pretty-print / evaluate expression AST
- Pattern: Visitor — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Pretty-print / evaluate expression AST
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Visitor: Pretty-print / evaluate expression AST

Real-World Scenario

- Running example: Pretty-print / evaluate expression AST
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```java
package com.buildwithwaqar.patterns.gof.behavioral.visitor.problem;

import java.util.*;

public final class NaiveAst {
  public static String show(String kind, int a, int b){
    if("add".equals(kind)) return "("+a+"+"+b+")";
    return "("+a+"*"+b+")";
  }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```
package com.buildwithwaqar.patterns.gof.behavioral.visitor.problem;

import java.util.*;
import java.util.concurrent.*;
import java.util.concurrent.atomic.*;
import java.util.function.*;

sealed interface Expr permits Num, Add, Mul { <R> R accept(Visitor<R> v); }
record Num(int v) implements Expr { public <R> R accept(Visitor<R> v){ return v.visit(this); } }
record Add(Expr l, Expr r) implements Expr { public <R> R accept(Visitor<R> v){ return v.visit(this); } }
record Mul(Expr l, Expr r) implements Expr { public <R> R accept(Visitor<R> v){ return v.visit(this); } }
interface Visitor<R> { R visit(Num n); R visit(Add a); R visit(Mul m); }
final class Pretty implements Visitor<String> {
  public String visit(Num n){ return Integer.toString(n.v()); }
  public String visit(Add a){ return "("+a.l().accept(this)+"+"+a.r().accept(this)+")"; }
  public String visit(Mul m){ return "("+m.l().accept(this)+"*"+m.r().accept(this)+")"; }
}
final class Eval implements Visitor<Integer> {
  public Integer visit(Num n){ return n.v(); }
  public Integer visit(Add a){ return a.l().accept(this)+a.r().accept(this); }
  public Integer visit(Mul m){ return m.l().accept(this)*m.r().accept(this); }
}
```

Runnable main

```
package com.buildwithwaqar.patterns.gof.behavioral.visitor.problem;
import java.util.*; import java.util.concurrent.*;
public final class ProblemDemo {
  public static void main(String[] args) throws Exception {
    System.out.println("=== Visitor: AST pretty-print / eval ===");
    System.out.println("--- naive ---"); System.out.println(NaiveAst.show("add",2,3));
System.out.println("--- visitor ---");
Expr e=new Mul(new Add(new Num(2), new Num(3)), new Num(4));
System.out.println(e.accept(new Pretty()));
System.out.println("value="+e.accept(new Eval()));
  }
}
```

Walkthrough Output

```
=== Visitor: AST pretty-print / eval ===
--- naive ---
(2+3)
--- visitor ---
((2+3)*4)
value=20
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Visitor.

Related Patterns

- Contrast Visitor with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Visitor → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.